CHAPTER 1

SYSTEM DESIGN CASE STUDIES

1 How Spotify Wrapped Scales for 7M Users 2
2 How Amazon Scales for Prime Day . 9
3 6 Tips from a System Design Expert: How Twilio Builds for Scale 16
4 Dropbox System Design 19
5 Ticketmaster System Design 27

Explore System Design case studies that reveal how top tech companies build scalable, resilient, and high-performance platforms.

1 How Spotify Wrapped Scales for 7M Users

Every year-end, Spotify delivers its users a gift: a beautifully personalized summary of their listening habits. From your top songs to your most-streamed genres, Spotify Wrapped transforms your music data into an engaging, shareable story.
Spotify Wrapped is also a powerful product offering that users wait for each year.
The numbers speak for themselves:

- 100 million+ shares on social media in 2022[1]
- 20% increase in Spotify downloads in 2020[2]
- 602 million users engaged across 184 countries (as of 2023)[3]

But Spotify Wrapped is more than just a successful marketing campaign; it's a time capsule of your year in music, powered by data science, machine learning, and immaculate System Design.
Behind every Wrapped recap is a robust architecture that processes petabytes of data with precision and speed. This means that engineers are working diligently to ensure Wrapped is seamless, scalable, and always ready for millions of users worldwide.
Let's explore the inner mechanics that make Spotify Wrapped work like clockwork. We'll cover:

- Challenges for scaling Spotify Wrapped
- System Design of Spotify Wrapped
- 4 engineering lessons we can learn from Spotify

Let's dive in.

1.1 Getting to know Spotify Wrapped

Spotify's scalable System Design enables the Wrapped campaign to reach millions without a hitch, turning personal listening data into a global phenomenon. By seamlessly scaling to meet huge surges in demand, Spotify ensures that each Wrapped experience is fast, personalized, and ready to share.

Wrapped over the years

Since its launch in 2016, Spotify Wrapped has added new layers of interactivity and personalization every year:

Estimating Wrapped users

We estimated user counts for 2024 based on past user data from Spotify

- 700 million total monthly active users on Spotify
  - Based on average growth of ~23%, 2019 to 2023
- 295 million users accessing Wrapped
  - Based on the average growth of 37.5%, from 2019 to 2022 (2023 data is undisclosed)

While not every user opens and accesses their Wrapped, Spotify creates the personalized Wrapped experience for each user (provided they meet simple eligibility criteria, such as minimum listening time).

Year	Spotify Wrapped Features
2016	• The first edition of Spotify offered basic stats, such as the top songs, artists, and genres, based on users' yearly listening habits
2017	• Expanded stats with more detailed insights, including top 5 artists, songs, genres, and the ability to share these stats on social media
2018	• Top artists, songs, genres • Added "Your Top Songs" playlist, allowing users to re-listen to their top songs of the year
2019	• Top artists, songs, genres • Introduced "Tastebreakers" playlist, which recommended new songs outside users' usual preferences • Added a slideshow for a more engaging experience • Introduced decade-based insights into users' listening history
2020	• Top artists, songs, genres • Focused on past listening patterns and added "Missed Hits" playlist • Introduced new stats like the number of new artists discovered, top podcasts, etc.
2021	• Introduced new interactive features like "Audio Aura." • The "2021: The Movie," which matched users' music to movie scenes • Introduced shareable "Wrapped Cards" for social media
2022	• Added personalized "Listening Personality" types and improved visuals • Introduced "Audio Day," offering a peak into evolving tastes based on preferences at different times of the day • Expanded on the slideshow, making it more dynamic and engaging
2023	• Enhanced "Listening Personality" insights with social sharing • Introduced custom storylines based on listening behavior • Upgraded interactive slideshow • Added "Me in 2023," which assigns users a unique listening character • Introduced "Sound Town," matching listeners to a city that reflects their music tastes • An "AI DJ" that guides users through their Wrapped with commentary on top songs and artists

Wrapped data

Spotify likely logs user data from January 1 through November 15 or 30.
Here are some insights on the Wrapped data that's collected:

- Top songs and artists are ranked by play count, not total listening time.
- Songs must be played for over 30 seconds to count in rankings.
- Only the first 10 songs in the top 100 playlists are strictly sorted by play count.

1.2 Challenges of scaling Spotify Wrapped

Processing data and creating Wrapped for 700 million users requires a scalable and robust architecture to process data from the year-long music history of their users. Spotify must manage millions of simultaneous streams, store and deliver petabytes of data, and recommend personalized content, all with low latency and high performance.
The engineering team faces several challenges when ensuring a seamless Spotify Wrapped user experience:

Scalability and resource management

Scalability is crucial amid Wrapped, as the surge in user engagement and social sharing can overload the system. Maintaining scalable, serverless, and auto-scaling solutions is critical, but these must be optimized without overloading and increasing costs.

Data volume and processing

Spotify handles an enormous amount of data, especially historical data, across hundreds of millions of users. Processing this data in batch jobs to compile Wrapped insights while simultaneously managing real-time data flows for recommendations requires highly efficient data pipelines and storage solutions, like data lakes and distributed storage.

Personalization complexity

Wrapped's success hinges on hyper-personalized insights, which require complex machine learning models trained on massive datasets. Scaling these models while avoiding latency issues is challenging, but advanced machine learning models can help optimize this process.

Cost management

Efficiently managing cloud resources during Wrapped's annual spike is key to balancing performance and costs. One may do this by integrating Wrapped calculations into existing data pipelines used for real-time recommendations.

Data compliance

Handling user data requires compliance with regulations like GDPR and CCPA. Spotify can ensure data privacy while maintaining low-latency data delivery through edge computing and real-world distributed systems.

1.3 Scalability techniques for Spotify Wrapped

Let's see how Spotify's scalability techniques help address these challenges.

Technique	Description	Challenges Addressed
Multi-tiered storage	Cloud-based tiered storage efficiently stores vast amounts of historical user data and Wrapped results.	Data volume & processing, cost management
Horizontal scaling	Adds servers instead of upgrading existing ones, enabling Spotify to handle massive concurrent user demand.	Scalability & resource management, availability
Serverless and auto-scaling	Uses serverless architectures and auto-scaling (e.g., AWS Lambda, GCP) to dynamically allocate resources as demand spikes.	Scalability, cost management
Data processing using the data lake	Processes user history and engagement data in a data lake or warehouse to manage high-volume batch processing needed for Wrapped.	Data volume & processing. personalization complexity
Real-time processing	Uses tools like Kafka and Spark to continuously process user data, ensuring real-time insights are available.	Data volume & processing. personalization complexity
Edge computing	Caches content closer to users on edge servers, reducing latency and handling regional load effectively during Wrapped access.	Data compliance, scalability & resource management
Monitoring and auto-recovery	Implements real-time monitoring tools like Grafana and failover mechanisms to detect and recover from issues quickly.	Scalability & resource management, availability

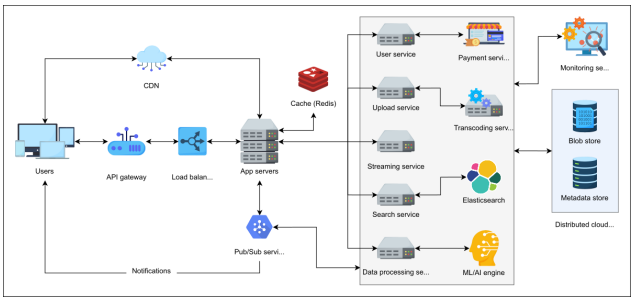


Figure 1.1: An overview of Spotify's scalable System Design

1.4 Spotify System Design and workflow

Spotify's system design ensures seamless streaming, user interactions, and personalized features, such as Wrapped. The architecture is designed for scalability and high availability, efficiently handling millions of simultaneous requests.

Key system components and services

Here's how Spotify processes user requests and ensures scalability:

- **API Gateway:** Acts as the entry point, authenticating user requests.
- **Load Balancer:** Distributes requests evenly across application servers to handle large volumes of traffic.
- **Messaging Queue:** User interactions (e.g., playing songs, creating playlists) are sent to a queue (like Pub/Sub or Kafka). This queue distributes data to various microservices for tasks such as generating recommendations or creating wrapped summaries. This allows the data to be processed asynchronously, improving scalability and availability.

This asynchronous approach enhances scalability and availability, ensuring Spotify's system can handle traffic spikes and real-time demands.
Spotify's microservices architecture supports various tasks:

- **User service:** Manages user data, including preferences and subscriptions, with connections to a payment service for subscription verification.
- **Upload service:** Ingests new content from artists.
- **Transcoding service:** Converts uploaded files into streaming-compatible formats, storing them in cloud-based blob storage (and metadata into an SQL database).
- **Streaming service:** Delivers content to users via a content delivery network (CDN), minimizing latency.
- **Search service:** Enables fast lookups using Elasticsearch.
- **Processing service:** Powers recommendations and wrapped summaries using advanced machine learning models.
- **Monitoring service:** Monitors the overall system's health and alerts in case of errors, failures, etc.

Distributed databases

Spotify employs multiple database types:

- **Blob storage:** Stores tracks, podcasts, and audiobooks.
- **SQL databases:** Store user metadata, such as account details.
- **NoSQL databases:** Handle activity data, including listening history, playlists, and preferences.

System Design for Spotify Wrapped

Let's explore how data processing services handle such a massive amount of data at scale to create personalized Spotify Wrapped experiences.
Spotify uses the ETL (Extract, Transform, Load) process: **Extract** defines how data is collected, **Transform** covers how data is processed and turned into features, and **Load** specifies where data is stored for efficient retrieval. They also use reverse ETL to create Wrapped from the processed data.

Data collection or ingestion (Extract) A **data collection service** collects data from data resources (databases) and passes it to tools like Kafka or Pub/Sub to stream and make it available for immediate processing.

Data processing (Transform) The data from the ingestion layer is fed to the processing layer, where the batch processor runs on massive data, aggregating users' year-long listening tasks and generating insights. Spotify utilizes Google Cloud Bigtable to efficiently manage its extensive time-series data and user listening history, optimizing it for rapid data aggregation over specific time frames.
In 2019, Spotify's use of Bigtable and BigQuery for data processing resulted in processing 5 times the data while reducing the overall cost by 25 percent.
Spotify can quickly compile user-level insights by structuring data storage to minimize shuffling (reducing the need to move data between nodes, which can be time-consuming and resource-intensive) during processing.

> **Note:** The following illustration is an in-depth exploration of how data processing services process data and transform it into a personalized Wrapped.

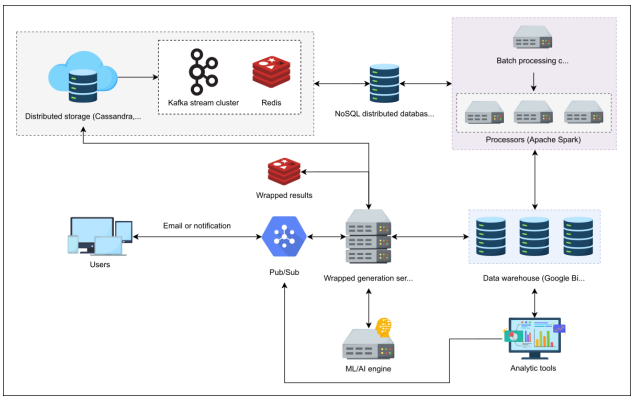


Figure 1.2: A detailed design of data processing for Spotify Wrapped

Data warehousing (Load) Apache Spark and other big data frameworks process this data at scale, and the results are stored in data warehouses, such as Google BigQuery.

Wrapped creation and personalization (Reverse ETL) Finally, data visualization tools and services aggregate this processed data, allowing Wrapped summaries to be sent to the user in real time through APIs. Cloud services ensure low latency, high availability, and scalability across Spotify's global infrastructure.
The Wrapped summaries are sent to users via Email or in-app notifications through the Pub/Sub service.

> **Note:** Spotify Wrapped is all about personalization, done by utilizing advanced machine learning algorithms. The ML engine uses collaborative filtering(A filtering technique to recommend content based on the behavior of similar users or items.), content-based filtering(A filtering technique to recommend content based on similarities between items or content based on metadata and content features.), and a hybrid model, mostly the best of both, to generate personalized Wrapped for each user.

For 2019 Wrapped, Spotify processed decade-long user data utilizing Bigtable. A similar data processing pattern for a year-long dataset is shown below:

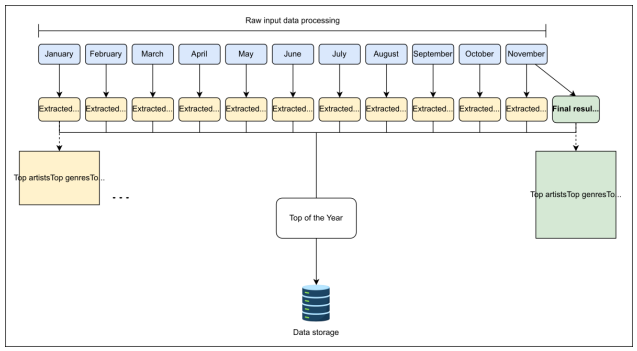


Figure 1.3: The architecture of data pipelines [source: Spotify]

Front-end animations in Spotify Wrapped

The front-end experience for Spotify Wrapped plays a crucial role in driving user engagement. The design of the Wrapped interface transforms raw data into fun, shareable content.
Spotify Wrapped's front-end elements include:

Visual and interactive features

- **Personalized visualizations:** Insights are displayed as animated reels or cards.
- **NLP-powered content:** Uses natural language processing to generate captions and labels for animations.

Feature highlights

- **Audio Aura (2021):** Colors representing listening intensity for different genres.
- **Sound Town (2023):** Mapped users' tastes to fictional cities, creating playful, shareable visuals.

These interactive features enhance user engagement, turning data into delightful experiences.

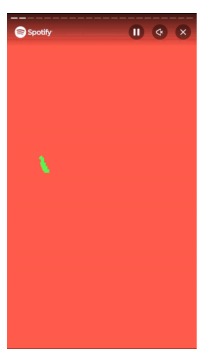


Figure 1.4: 2023 Sound Town [Source: Spotify]

1.5 What we can learn from Spotify

Here are 4 key takeaways from Spotify's approach to delivering a seamless Wrapped experience each year:

1. A robust, scalable System Design is the backbone of Wrapped. It handles huge data volumes by separating real-time and batch-processing content, ensuring fast data access and reliable yearly insights.

2. Using solutions like Bigtable and BigQuery, Spotify minimizes data shuffling and enables efficient aggregation, providing users with quick, user-level insights for millions.

3. Advanced machine learning models help Spotify deliver Wrapped's unique, personalized insights by analyzing patterns in listening data.

4. By employing auto-scaling and load balancing, Spotify can smoothly manage the surge in Wrapped engagement.

1.6 What's next for Spotify Wrapped?

As Spotify Wrapped continues to scale year after year, it offers a glimpse into the complex System Design that powers real-world applications at a massive scale.
Spotify's developers continually add new features and insights to enhance the user experience, making it challenging to predict what's coming next. However, we can expect AI to level up the Wrapped experience through features like:

- Interactive, real-time playlists that evolve based on a user's Wrapped experience.
- AI-driven music DNA visualizations breaking down listening habits into dynamic, shareable formats.
- Leveraging GenAI to create unique soundtracks for users, blending favorite genres, moods, and artists into a custom composition.

Spotify Wrapped highlights the importance of approaches like cloud-based data pipelines, advanced machine learning models, and auto-scaling for cost savings in System Design.
Disclaimer: All technical information and design insights provided in this lesson are curated by our System Design experts to the best of their knowledge and based on available resources, including insights from Spotify's engineering blogs. While we strive for accuracy, some details may vary from Spotify's actual implementations and are intended for educational purposes only.

2 How Amazon Scales for Prime Day

Imagine this: You've been eagerly counting down the days until Amazon Prime Day, waiting to catch that perfect deal. Finally , the moment arrives. You rush to the site... only to face error messages and pages that won't load. Frustrating, right?
This was the reality for millions of shoppers during Amazon's 2018 Prime Day outage. Let 's unpack what actually happened from a System Design perspective

2.1 Amazon Prime Day outage

Amazon experienced a significant outage during its 2018 Prime Day event. The outage lasted for several hours and had a noticeable impact on the shopping experience of millions of customers, as well as on the business itself.
Impact: The main Amazon website and mobile app were affected, with users reporting issues accessing product pages, completing purchases, and loading the home page. According to the report, Amazon lost over $100 million in sales during the downtime, an estimated $1.2 million per minute.

Causes of the outage

While Amazon has not disclosed all the technical details, we can analyze the causes with industry knowledge and engineering principles based on a CNBC report, which includes the following key factors to failure:

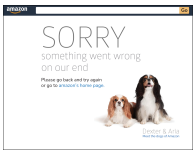


Figure 1.5: Amazon's main page on Prime Day 2018

1. The first and most important factor is the higher-than-expected spike in traffic, which exceeded Amazon's capacity despite the preparation. This observation is evident from Amazon's steps, which included falling back to a simpler frontend page and blocking international traffic.

2. Although Amazon employed a robust microservices architecture, it may have faced issues where service dependencies led to cascading failures. This issue was also reported for other Amazon services, including Twitch, Alexa, and Prime Now.

3. While Amazon utilizes autoscaling to handle surges in traffic, delays occurred in scaling up resources due to configuration issues with Amazon's systems and services. This forced Amazon to manually deploy servers to meet the traffic load, which cost time and money.

The CNBC report indicates that the surge in traffic was one of the primary factors Amazon couldn't handle, as it made the service unavailable. The system received approximately 63.5 million requests per second on its storage and computational service (Sable).
In this blog, I'll explore the technical intricacies of scaling for traffic during high-demand periods, using Amazon as our case study.
We'll break down the strategies and approaches that ensure a smooth service, regardless of the number of people searching for products, clicking "add to cart", or placing orders. Whether you're a tech enthusiast, an e-commerce entrepreneur, or just someone who wants to understand what happens behind the scenes during these mega-sale events, this journey into Amazon's world of scalability will be enlightening.
Let's explore the importance of availability bound to scalable systems.

2.2 Importance of scalability and availability

Scalability is the secret sauce allowing companies like Amazon to handle massive traffic spikes without breaking a sweat.
It's about having the infrastructure to flexibly expand and contract in response to demand. The 2018 Prime Day outage wasn't just a hiccup; it was a wake-up call that even the best in the business can stumble. But many people don't realize that ensuring scalability isn't just about handling spikes in traffic; it's intrinsically linked to availability.
Let's look at how Amazon processes a user's request.
A single API request, such as purchasing an item or performing a search, is routed to multiple backend services, including inventory, payment, shipping, indexing, and recommendation. Each service must handle part of the request to fulfill it successfully.
When scaling, all these backend services need to be scaled together to manage increased load effectively, as seen in the purchasing and search examples below:

Availability, keeping the services up and running, is the backbone of any online business. A few minutes of downtime can translate into millions of dollars in lost sales, and a spoiled reputation is additional. Availability is important for the success of a service and a seamless user experience. This means that users can access the service and perform the intended operations at any time. In service level agreements (SLAs), availability is defined as a percentage of time, usually represented by 9s, such as 99.999% uptime of the service.

> **Note:** When we scale a system to handle a surge in demand, it is also crucial to ensure service availability. No matter how much we scale, if we don't have replicas or failover mechanisms to ensure availability, the service will likely fail, and vice versa.

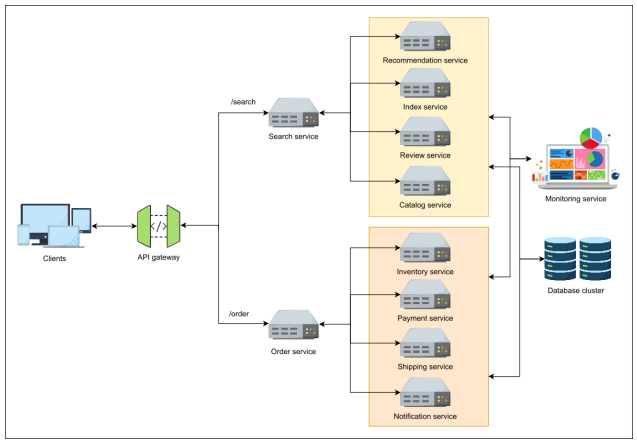


Figure 1.6: A depiction of fanning out requests to multiple subservices

What are the optimal 9s of availability?

When deploying services, an uptime of 99.999% (five 9s) is assumed as the perfect uptime. This allows for only 0.001% of downtime. Let 's look at some calculations to see what that entails:

$$24\ hours \times 365\ days = 8760\ hours/year$$

$$8760\ hours \times 60\ minutes = 525600\ minutes/year$$

$$525600\ minutes \times \frac{0.001}{100} = 5.26\ minutes$$

A downtime of 0.001% equals less than 6 minutes; a service will be down for around 6 minutes *in an entire year*. Now , 6 minutes in a year doesn't sound like much, but for services like Amazon, a minute can cost them around a million in sales, especially on events like Prime Day.
Let's estimate Amazon Prime Day's revenue for 2024 by examining its sales history. Take a look at the graph below:

Amazon Prime Days revenue each year

[Chart data presented in table format]

Year	Revenue ($ billion)
2015	0.9
2016	1.45
2017	2.33
2018	4.15
2019	7.16
2020	10.39
2021	11.19
2022	12.09
2023	12.9
2024	14.3

Table 1.1: The expected revenue of Amazon in 2024 (in green).
Note: Values are in Billion

Note: The expected revenue of Amazon Prime Day 2024 is based on the average annual revenue difference ($1.43 billion) from the above data.

Based on the expected revenue for 2024, Amazon is projected to generate approximately $5 million per minute in sales. Now , let's see how the 9s of availability affect the downtime and, hence, the loss in revenue of Amazon Prime Day:

Nines (9s) of availability

Availability	Downtime Per Year	Downtime Per Month	Downtime Per Day	Revenue Loss Per Day
1 nine —- 90%	36.5 days	72 hours	2.4 hours	$720 million
2 nines —- 99.0%	3.65 days	7.20 hours	14.4 minutes	$72 million
3 nines —- 99.9%	8.76 hours	43.8 minutes	1.46 minutes	$7.3 million
4 nines —- 99.99%	52.56 minutes	4.32 minutes	8.64 seconds	$0.72 million
5 nines —- 99.999%	5.26 minutes	25.9 seconds	0.86 seconds	$0.072 million

Table 1.2: Nines (9s) of availability

The above back-of-the-envelope calculations estimate the loss if Amazon faces downtime on the next Prime Day (2024).

> **Note:** The problem with achieving five or six nines of availability is that it costs a lot of money and effort. The good thing is that you don't need to have five or six nines of daily availability, except for days like Amazon Prime Day. The cost of achieving five 9s is so high that the returns do not justify the investment.

Isn't Amazon Web Services (AWS) good enough?

Amazon Web Services (AWS), Amazon's cloud platform, offers a powerful suite of services designed to keep modern applications resilient and highly available.
From Amazon EC2 Auto Scaling, which automatically adjusts compute capacity, to Elastic Load Balancing (ELB), which distributes incoming traffic across healthy targets, AWS provides the foundational tools needed to handle massive traffic surges—such as those seen during events like Amazon Prime Day.
However, even the most advanced cloud infrastructure can experience issues.
In many cases, outages occur not because the tools are insufficient, but because of how they are configured, monitored, and managed. AWS services provide the capabilities, but achieving true reliability requires engineering teams to configure those services correctly, establish clear scaling policies, design for failure, and continuously test their systems.
This serves as an important reminder: AWS offers the building blocks, but availability and scalability ultimately depend on sound architectural decisions. To understand how Amazon achieves this at scale, let's examine the core principles behind designing systems for high availability and elastic scalability.

2.3 An overview of Amazon's e-commerce infrastructure

Amazon started as an online bookstore and has since evolved into the world's largest e-commerce platform, offering a vast array of products.
Let's see how the Amazon e-commerce platform works for selling and purchasing goods. The Amazon system must be highly scalable and available to handle millions of transactions per day, particularly during events like Amazon Prime Day.
Yes, you guessed it right; it should also be performant and consistent, which, for the time being, will be considered secondary metrics. The primary metrics are scalability and availability. Let 's have a walkthrough of the Amazon back-end infrastructure as given in this high-level design:

Initially, the users can visit the homepage to search for an item of interest.
The search service utilizes the Elasticsearch system for several key advantages, including performance, scalability, and full-text search features. At the back of the homepage is a recommendation service that provides personalized recommendations to users based on their purchase history or search activity.
If the user is new, they are shown some trending products on the home page.
Once the users select items and add them to the shopping cart, the cart service is triggered at the backend, which keeps the items in the relevant databases. The order service handles incoming orders from users and

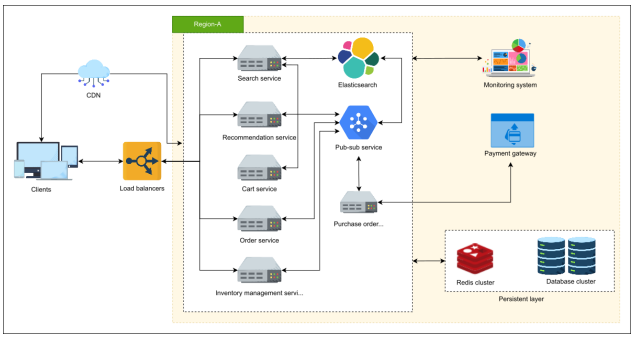


Figure 1.7: A high-level design of Amazon's e-commerce system

stores detailed information about the order, including customer and product information, in the databases. This service is responsible for capturing the delivery address and the relevant information.
When users attempt to pay and purchase an item, the purchase order is invoked, which interacts with the payment gateway, which, in turn, is responsible for collecting the due amount from the customer's credit card or account. The pub-sub system decouples various services, allowing them to communicate asynchronously.

2.4 Techniques to ensure scalability and availability

Let's examine some techniques that can be implemented to ensure the scalability and availability of the system. The most prominent techniques are:

- **Database replication, distributed caches, and backups:** Database replication techniques ensure data availability and significantly enhance scalability. Using the right techniques to store and retrieve data is essential to maintaining seamless service continuity. This includes replicating data across multiple databases and regions, partitioning data according to application needs, and maintaining different storage solutions for different data types. Techniques like distributed caching enhance performance by storing frequently accessed data closer to where it is needed, efficiently managing a large volume of incoming requests. Additionally, incorporating regular backups is crucial for disaster recovery. While these strategies require cost and effort, they are essential for maintaining the system's reliability and readiness for unexpected events.

- **Load balancing and autoscaling groups:** This is a no-brainer. Implementing load balancing involves distributing incoming traffic among targets, such as containers, servers, and data centers. This prevents overwhelming a single server and allows another to step in if needed. Similarly, autoscaling groups enable a system to adjust the number of active instances based on demand, leading to enhanced availability, performance, and user experience.

- **Redundancy of services:** By duplicating critical services across multiple zones and regions, a system can ensure that even if a service in one region fails, another can take over seamlessly. This redundancy minimizes downtime and latency, handles large requests efficiently, and maintains the platform's reliability, ensuring a consistent user experience.

- **Content Delivery Networks (CDNs):** Utilize CDNs to deliver content to users with low latency and high transfer speeds. By caching content at edge locations worldwide, they ensure that users receive data from the nearest server, reducing load times and improving the overall user experience. It also reduces the burden on the origin servers. CDNs are essential for efficiently handling large amounts of traffic and improving overall availability.

- **Monitoring and auto-recovery mechanisms:** Effective monitoring ensures a system's health and performance. Utilizing tools for real-time monitoring of system metrics (such as response time, resource utilization, access logs, temperature, and security events) and setting up automated alerts helps engineering teams promptly address potential issues. Implementing auto-recovery processes, such as

restarting failed instances or triggering automatic failovers, is essential for maintaining service continuity and minimizing downtime. These techniques are vital for proactive system management and ensuring reliable operation under varying conditions.

- **Testing:** Thorough load tests simulating peak traffic conditions are crucial. These tests should account for the worst-case scenarios, ensuring all systems can handle extreme loads.

You might think you can get by without the techniques above, and maybe you can, until that one bad day strikes. Being prepared can make all the difference. Trust me, you'll be grateful you invested in these strategies when that day comes.
In reality, ensuring a seamless experience, even on big days, requires considerable engineering brilliance and a commitment to consistent improvement. That 's only the first half of the problem. What happens if there is a failure? That's the second half! You need standby engineering teams on high alert with contingency plan(s) to ensure you recover quickly and successfully.

It is crucial to maintain standby engineering teams that are ready to respond to incidents or system failures at any time. These teams should be equipped with the tools and knowledge to quickly diagnose and resolve issues, ensuring minimal disruption to services. Having dedicated on-call engineers allows the system to maintain high availability and quickly restore services in case of unexpected failures.

Let's see what techniques Amazon uses to ensure scalability and availability in the following table:

Key takeaways from the Amazon outage

Amazon's 2018 Prime Day outage highlights several important lessons for managing high-demand periods for engineering teams and tech enthusiasts:

- Building a perfect system is impossible, but there is nothing wrong with striving to achieve a well-designed one. How do you achieve that? Keep your design as simple as possible and evolve it slowly to avoid unnecessary complexities.

- Blame the configuration management or load testing teams; systems inevitably fail, even among tech giants like Amazon, Meta, and Google. Surprisingly , if you visit independent services like Downdetector, you may find your favorite application struggling to provide service in some parts of the world. The key aspect here is preparation for such incidents, including mitigation techniques to employ before they occur, a contingency plan in place after they happen, and the lessons learned from past failures.

- Faulty capacity estimation is a primary reason why systems fail. You design systems and prepare for the traffic you estimate. A faulty estimation leads to a faulty design. Therefore, always base your math on meaningful assumptions and intelligent guessesto make informed design decisions.

Conclusion

A system like Amazon needs to analyze user behavior and usage patterns and predict traffic spikes.
This can be achieved via advanced analytics, where metrics such as peak viewing times, popular content, and regional user distribution are continuously monitored. Furthermore , traffic patterns can be predicted through big data analytics technology, and the system can be prepared for potential surges.
Simultaneously, monitoring systems can provide valuable insights into current usage and facilitate better resource management.
To handle peak loads effectively, it's essential to introduce autoscaling systems that adjust the number of servers according to demand. AWS utilizes autoscaling groups to automatically add or remove instances based on fluctuations in traffic.
The combination of predictive analytics with reactive scaling and load balancing will enable the system to manage peak loads optimally without sacrificing performance.

Strategies for Scalability and Availablity	**Amazon's Techniques and Services**
Database replication and backups	• **Amazon RDS** for multi-availability zone deployment • **Amazon DynamoDB** for fully managed multiregion, multi-active data replication • **Amazon Aurora** for synchronous replication for high availability within a region and asynchronous replication for cross-origin data redundancy • **Amazon DocumentDB** for multiprimary replication across multiple availability zones
Distributed cache	• **Amazon ElasticCache** is the distributed cache solution used for high performance, scalability, and availability
Load balancing and Autoscaling	• **Amazon Elastic** load balancing for distributing different types of traffic across various services • **Amazon EC2** autoscaling for automatically adjusting the number of EC2 instances • **Amazon ECS** and **EKS** autoscaling for containerized applications
Content Delivery Networks (CDNs)	• **Amazon CloudFront** for placing the content near users
Monitoring and autorecovery mechanisms	• **Amazon CloudWatch** for monitoring and observing the services • **Amazon EC2 Auto Recovery** enables automatic recovery of EC2 instances • **AWS Systems Manager** provides a unified user interface for managing AWS resources • **AWS Health** provides personalized alerts and recovery guidance in the event of failure
Testing	• **Amazon GameDay** simulates failures to prepare for unforeseen situations

3 6 Tips from a System Design Expert: How Twilio Builds for Scale

The importance of System Design in the current tech landscape cannot be overstated.
Companies require software systems that are resilient, reliable, performant, and scalable to effectively serve customers and achieve their business objectives. As a result, System Design has become essential to the software development process.
This lesson draws its insights from Mark Gilbert, former VP of Product Management at Twilio, who shared his experiences scaling the platform. Whether a learner is new to System Design or already working on distributed systems, these fundamentals are critical for career advancement.

3.1 Twilio and its systems

Twilio is a Communications Platform as a Service (CPaaS) company that enables businesses to easily communicate with customers and others via text messages, emails, and calls.
The company is well-known for providing programmable communication tools on various communication channels using web service APIs. During his tenure, Mark Gilbert oversaw the Twilio Super Network and onboarding of phone numbers.
With the Super Network, Twilio combined the communication networks of multiple global carriers to support channels like voice, text messaging, email, and video reliably.
Getting customers onto the Super Network involved onboarding numerous phone numbers and working on international infrastructure. Regarding Twilio's tech stack, different teams would work on different system parts. Teams often gravitated toward their preferred programming languages, but then Twilio began pushing to migrate most of the systems to Java, valued as a type-safe language.
For data storage, Twilio used multiple solutions.
When it started, Twilio had to build much of its infrastructure, including components that are now available off-the-shelf from cloud providers like Microsoft, Amazon, and Google. Now , the scale of the systems at Twilio is quite large. On a big day, Twilio sends single-digit billions of emails and hundreds of millions of text messages.
Traffic is spiky, and the distribution of all those emails and messages is not even throughout the day.
Because of this and other challenges particular to Twilio's technology, scaling was a continuous challenge and top concern for everyone at Twilio as it built out its systems. To meet that challenge, Twilio paid a lot of attention to System Design principles and carefully constructed its architectures.

3.2 Six takeaways about System Design and architecting for scale

The continuous challenge of scaling at Twilio, as described in the case study, led to the following six key takeaways.

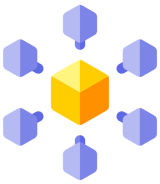
1. Add capabilities to existing architectures when possible

Leaders and engineers at Twilio debated daily about how to improve systems.
All that discussion was for a good reason: Twilio's customer growth was large, and making changes with all those customers on the system was a major challenge. It became essential to determine the safest and most efficient ways to improve.
To determine system requirements, the teams went back to basics, looking at expected traffic scenarios,

doing the math, and penciling out: *what's going to get there, what's not going to get there.* The real options involved adding role instances (nodes/servers) where they could. Sometimes it worked. Sometimes it worked, but inefficiently, and they had to look at why.

2. Sometimes adding servers is not the solution

Certain system challenges at Twilio defied the solution of adding extra servers.
For example, Twilio works with thousands of mobile operators and providers worldwide. A lot of these providers don't have modern systems that scale. This would lead to problems with downstream capacity, as those providers' systems ran into problems when they got more messages than they could handle.
Some providers might drop the messages altogether, while some would let them go later.
And they didn't return clean errors. All these different kinds of "weird behaviors" can happen. In these situations, adding servers didn't solve the problem. In addition, some of Twilio's systems created a lot of communication between servers.
Ultimately, capacity issues would occur.
The amount of "chatter" might become so high that adding role instances wouldn't help scale the system. Then , the answer involved having programmers review the communication, the services that created it, and the partitioning of the servers. To change those things while they were constantly operating was a fairly heavy lift.
When adding servers wouldn't do the trick, the next step was to rearchitect certain pieces of the system for scalability. This became a high-priority and time-intensive effort.



3. Optimize the number and size of microservices

Twilio used a microservices architecture.
As with many large-scale cloud-based services that emerged in the mid-2000s, Twilio employed microservices, although not to the extent they would have liked. Start -ups must experiment and revise their designs frequently initially. At this stage, Twilio had to weigh the potential benefits of a microservices architecture against the costs of later changing it.
The trick was finding a sweet spot: not too many microservices, but not too few.
If someone wanted to split an app's architecture into microservices to the utmost, it should be reconsidered. However, if someone has everything on one service, you may need to reconsider that approach. There were two main considerations when determining whether a system's microservices were getting too small or numerous:

- **Diagnostics and debuggability:** A plan is needed for investigating errors and failures.
- **Performance:** If service boundaries are not tightly coupled, performance issues will appear.

Trying to find the optimal number of microservices prematurely is challenging.
It is necessary to look at requirements for the front-end (responsiveness) and back-end (reliability), and try

to anticipate what services may require splitting up. Then , as the app receives more traffic, it can be seen with greater confidence which microservices are likely to have issues.

4. Plan around services' expected lifespans

To scale from prototype to production, one must pencil out how long components are expected to last. To do so:

- Break down the app's architecture into services: What are the pieces? How will they communicate? What are the resulting requirements?
- Figure out which services are planned to be kept and those expected to be replaced: Invest time into understanding how to scale services planned for consistent use. Put appropriate service boundaries around them. Don 't waste as much design time on pieces for which there is less confidence.

At Twilio, this was applied to a large communication service. The team identified that the underlying components (text messages and emails) would be consistent, so they invested more design effort to ensure they could scale. At the same time, they anticipated having to rewrite the user interface multiple times, so they invested less in architectural thinking.
The challenge lies in discerning which services are which, as expectations often go awry.

5. Companies will increasingly build more flexible systems

Amid growing dependence on the cloud and software as a service (SaaS), tech companies will increasingly build systems out of small services that can be deployed dynamically.
During the 2008-2010 period, many start-ups claimed to be using microservices, but in reality, they were also heavily relying on cloud infrastructure, such as AWS and Microsoft Azure. The cloud was fast to build on and quickly reached a reasonable scale.
As a result, they would end up with a "start-up ball of wax": systems they would continually have to claw pieces out of to scale differently.
Now companies are paying increasing attention to scaling, deploying into different regions worldwide, and breaking up services to run on different providers. They 're creating more thoughtful microservices and being careful about taking on dependencies.
Partly, this has resulted from shifts outside the tech industry, such as increased attention to data privacy, geolocation of data, and residency.
Companies will increasingly use a model that combines cloud provider services with services they integrate into their own infrastructure and deploy elsewhere. This approach scales more effectively and provides companies with greater control.

6. In technical interviews, give real examples and explain the thinking

Interviews at Twilio included questions assessing how well candidates understood System Design fundamentals and writing code.
But interviewers also drilled into candidates' experience working on systems, their reasoning behind decisions, and what they'd learned from their choices. Interviewers would continue to question candidates to determine whether they had truly done the work, how well they understood scale limitations, and what they would do differently in the future.

> *You can pretty soon tell whether someone understands the higher-level abstraction and whether the underlying 'how this works' is somewhat understood—Mark Gilbert*

For more experienced candidates, interviewers would look for designing and redesigning for scale in their backgrounds. Improving scaling on existing services is a measurably harder problem that engineers might face as they gain experience.

> *The black-belt ninja moves are taking large systems in flight and trying to scale those more—Mark Gilbert*

For candidates with fewer years of experience, questions relating to Twilio's values might be more critical. For example:

- Are you adding positive value?
- Are you curious?
- Are you learning?

Even candidates fresh out of school or self-taught would get questions about what they had built and how it had worked. Regardless of their experience, candidates who could provide concrete examples and explain their thinking stood to perform better.
The lessons from interviews at Twilio apply more generally:

- Show up to interviews prepared to discuss not only the projects worked on, but also the thought process and lessons learned from those experiences.
- Any project brought up should be one in which the candidate played a significant role and understands thoroughly.
- For the System Design Interview, be prepared to devise solutions to hypothetical problems.
- Ensure you have studied the company's values beforehand and be prepared to demonstrate alignment with them.

3.3 Conclusion

The six takeaways from Twilio's experience emphasize one central theme: thinking strategically about System Design has become vital in the modern tech landscape, from small start-ups to FAANG companies.

4 Dropbox System Design

Dropbox is a cloud-based file storage and synchronization service that allows users to store, share, and collaborate on files across multiple devices.
When a file is added or modified in a Dropbox folder, the system ensures that all connected devices and collaborators have access to the most up-to-date version. This requires efficient storage, synchronization, and metadata management, as well as secure authentication and access control.
Dropbox is one of several cloud storage and collaboration tools available today, alongside:

- Google Drive
- Microsoft OneDrive
- Box
- Jupyter

As of November 2023, Google Drive held the largest market share in cloud storage, with Dropbox in second place. Dropbox 's continued adoption demonstrates its focus on reliable synchronization, simplicity, and scalability, particularly for collaborative workflows.

Market Share of File Sharing Systems

[Chart data presented in table format]

Label	Percentage Market Share
Google Drive	31.19
Dropbox	18.61
OneDrive	14.84
Box	12.96
Jupyter	8.45
Other	13.95

Table 1.3: The market share of top 5 file synchronization and sharing services

4.1 Requirements

Defining requirements is a crucial first step in designing a system like Dropbox. Requirements are typically categorized as either functional or nonfunctional, providing a clear scope for the design process.

Functional Requirements

- Upload files
- Download files
- Share files/directories
- Create and delete directories
- Synchronization upon file changes
- Version control for files
- User authentication and access control

Nonfunctional Requirements

- Durability and reliability
- Availability
- Scalability
- Security
- Low-latency file retrieval

Clarifying requirements is also essential in System Design interviews, as it demonstrates a structured approach to understanding the problem scope. For example, distinguishing between functional and nonfunctional requirements provides guidance for both the backend architecture and the client-side design.

After requirements are established, it is necessary to estimate the system resources required to support expected user activity. This includes determining storage, bandwidth, and server capacity.

4.2 Resource estimation for Dropbox architecture

Resource estimation involves calculating storage, bandwidth, and server requirements based on projected user activity. For this analysis, assume Dropbox has 50 million paying users.

> **Note:** According to Backlinko(https://backlinko.com/dropbox-users), Dropbox had approximately 18.22 million paying users as of Q2 2024. Designing for 50 million users allows planning for future growth.

Storage estimation

If each user is allocated 2 TB of storage, the total storage requirement is calculated as:

$$\begin{aligned}\text{Total storage} &= \text{Number of users} \times 2 \text{ TB} \\ &= 50 \times 10^6 \times 2 \text{ TB} \\ &= 100 \times 10^6 \text{ TB} \\ &= 100 \text{ PB}\end{aligned}$$

Thus, the system would require approximately 100,000 petabytes of storage to accommodate 50 million users.

Bandwidth estimation

Assuming that each user consumes 1 GB of bandwidth per day, the total bandwidth requirement can be estimated as:

$$\begin{aligned}\text{Total bandwidth} &= 50 \times 10^6 \times 1 \text{ GB} \\ &= 50 \text{ EB/day} \\ &= \frac{50 \text{ EB}}{24 \times 60 \times 60 \text{ s}} \\ &\approx 579 \text{ GB/s} \\ &= \frac{579 \text{ GB/s}}{8} \\ &\approx 72.5 \text{ Gbps}\end{aligned}$$

This calculation provides a baseline for network infrastructure to support daily user operations.

Server estimation

To estimate server requirements, first calculate the queries per second (QPS). Assuming each active user sends 5 requests per minute:

$$\text{QPS} = \frac{50 \text{ M} \times 5 \text{ requests}}{60} = 4.2 \text{ M}$$

> **Note:** According to the back-of-the-envelope estimations, a 64-core server executes approximately 64000 requests per second.

Considering that a 64-core server can handle approximately 64,000 requests per second, the required number of servers is:

$$\text{Number of servers} = \frac{4.2 \text{ M}}{64000} \approx 66 \text{ servers}$$

So, to summarize, for 50 M users, the following is the estimation for different resources:

- Estimated storage = 100 EB
- Estimated bandwidth = 72.5 Gbps
- QPS = 4.2 M
- The number of servers = 66 servers

You can estimate these resources for different numbers of users in this calculator:

These estimations provide a foundation for designing storage, networking, and server infrastructure to support anticipated user demand.

4.3 High-level design of Dropbox

The high-level architecture of Dropbox consists of multiple interacting components and services that manage file storage, metadata, synchronization, and access control. Key components include:

- **Metadata service:** Maintains information about files, directories, and user activity.
- **Chunk service (chunker):** Responsible for splitting files into chunks and handling their storage and retrieval from cloud storage.
- **Metadata database:** Stores metadata, including file paths, versions, ownership, and access permissions.
- **Cloud storage:** Provides scalable, distributed storage for encrypted file chunks.
- **Synchronization service:** Coordinates file updates across multiple devices, ensuring consistency and notifying collaborators of changes.

The overall system ensures that when a user uploads or modifies a file, the metadata service, chunk service, and synchronization service collaborate to update cloud storage and propagate changes to all devices associated with the user's account.
Replication and redundancy mechanisms ensure high availability and durability.

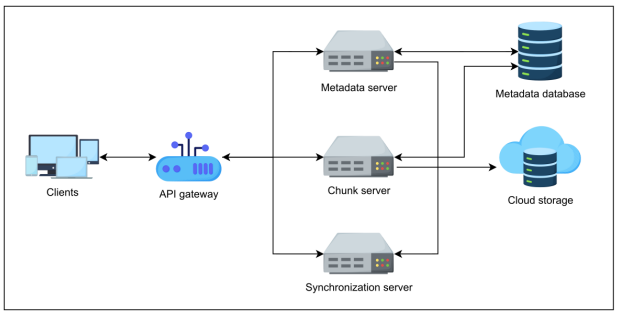


Figure 1.8: A high-level design of Dropbox system

This high-level view provides a foundation for the detailed design, where each functional requirement is mapped to specific components and interactions.

4.4 Detailed design of Dropbox

Let's dive into Dropbox's detailed System Design by starting with each functional requirement. We 'll discuss how to achieve each functionality step by step and the architectural changes necessary to support it.

File upload and download

The upload process begins when a user places a file in the client application's Dropbox folder. The file is divided into smaller chunks, typically 4 MB each. Each chunk is hashed using SHA-256, producing a unique identifier, and encrypted with AES-256 before storage. The encrypted chunks are distributed across multiple servers to ensure high availability.

The client application coordinates with the chunk server, synchronization service, and cloud storage to manage uploads, downloads, and synchronization. During downloads, encrypted chunks are retrieved, decrypted locally, and reassembled. Caching and compression techniques are used to enhance transfer speeds and minimize server load.
The following pseudo-code shows the chunking process and computing their SHA-256:

Pseudo code for creating chunks and computing their hashes using SHA-256

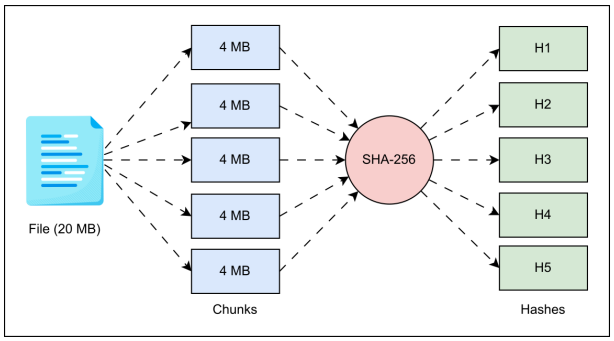


Figure 1.9: The process of dividing a file into chunks and then computing hashes

```
1  IMPORT SHA256 from cryptography library
2
3  FUNCTION create_file_chunks_with_hash(file_path, chunk_size):
4      INITIALIZE empty list called 'chunks'
5      INITIALIZE empty list called 'hashes'
6
7      OPEN file at file_path in binary read mode
8      SET file_position to 0
9
10     WHILE file_position is less than file size:
11         READ next chunk of chunk_size bytes from file
12         APPEND the chunk to 'chunks'
13
14         COMPUTE hash of the chunk using SHA256
15         APPEND the hash to 'hashes'
16
17         INCREMENT file_position by chunk_size
```

Explanation of the above pseudo-code:

- **Line 1:** Load the SHA256 library.
- **Lines 3–20:** Define a function that takes two parameters as input: the file path and the chunk size. This function returns a list of chunks and their hashes.
- **Lines 7–17:** Open a file located at file_path for reading in binary mode, and the file_position is set to 0 to ensure that the entire file is processed chunk by chunk. Next, the file is read chunk-wise using a loop, and each chunk is appended to a list. Also, the SHA-256 hash is computed for each chunk and appended to the hash list.
- **Lines 19–20:** Once the file is read, it is closed, and the lists of chunks and hashes are returned.

The chunk server validates each chunk by comparing its hash to existing entries. If a chunk already exists, it is not stored again. During downloads, chunks are retrieved and reassembled before delivery to the client.

File sharing

When a user shares a file or folder, the sharing service generates a unique link or sends invitations to designated collaborators. The service enforces permissions such as *read-only* or *can edit* to control the level of access granted to each participant. All associated metadata, including ownership details, access control lists (ACLs), creation and modification timestamps, and permission settings, is stored and managed within the metadata service.

User authentication and access control

In a system like Dropbox, user authentication and access control are crucial to ensuring that only authorized users can access files and perform specific actions. These tasks are managed through several components, including the authentication server, authorization server, and access control mechanisms.

- **Authentication:** Supports username/password, OAuth 2.0, and multi-factor authentication (MFA). MFA adds an additional verification layer, such as a one-time password.
- **Authorization:** Manages access based on role-based access control (RBAC), assigning permissions according to user roles (e.g., owner, editor, viewer).

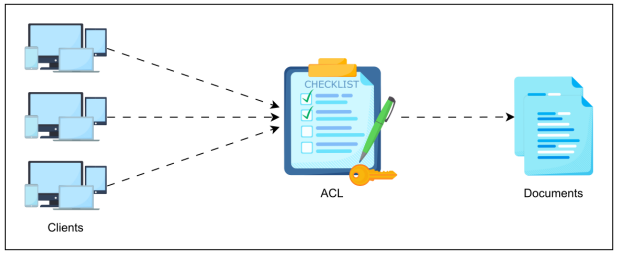


Figure 1.10: Access control in Dropbox

Additionally, the access control list (ACL) tracks the permissions assigned to each user or group for specific files and folders, ensuring that only authorized users can view, edit, or share content.

File synchronization

File synchronization in the system involves the interaction of different components, including the Dropbox client, metadata service, chunk service, and synchronization service.
The client's responsibility is to monitor changes in local files for changes and sync them to the cloud, communicating with the aforementioned services. In the file syncing process, the primary actor is the sync engine, a process that runs locally on the user's device.
It breaks files into chunks, computes hashes, detects changes, and initiates the upload process.
It is necessary to remember the distinction between a sync engine and a synchronization service. The synchronization service sits between the user's device and backend storage. It ensures that changes made on one device are reflected across all devices linked to the same account and are visible to the file's collaborators.
For file synchronization, a **block-level sync algorithm** can be used to improve efficiency.
This method divides large files into smaller chunks, enabling faster uploads and downloads. When a file is modified, the client computes and compares chunk hashes against those stored on the chunk server. Only the chunks that have changed are uploaded.
During download, the client retrieves just the updated chunks and then reassembles the file locally.
This approach significantly reduces data transfer and improves performance for large files—such as videos or database backups—where small edits would otherwise require reuploading the entire file. The following illustration demonstrates how only the modified chunk is uploaded to cloud storage.
The synchronization service then sends both the update and a corresponding notification to collaborators through the pub/sub (publisher/subscriber) system.

When it comes to handling large files and network interruptions, the block-level sync algorithm provides resilience and efficiency.
For large files, the client can upload or download chunks independently, meaning if there's a network interruption, the sync engine can resume from the last successfully uploaded or downloaded chunk rather than restarting the entire process.
This chunked approach is critical for users on unreliable networks or with bandwidth constraints. Dropbox also uses delta synchronization(Delta synchronization identifies and synchronizes the actual changes (deltas) between file versions, often at a finer level (e.g., byte or line-level). It 's more granular and flexible than

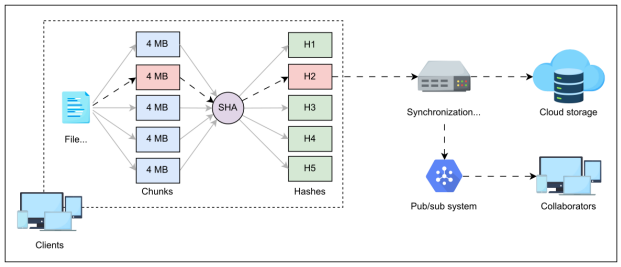


Figure 1.11: The modified chunk is uploaded to the cloud storage and sent to other clients via pub/sub system

block-level sync, which divides files into fixed-size blocks and syncs only modified blocks. Delta sync offers more precision, while block-level sync is simpler and more rigid.), ensuring that only modified chunks are synced, further optimizing the process.

> **Note:** Dropbox uses an indexer to manage and search the stored files metadata, enabling quick retrieval and synchronization across devices.

> **Quiz: Question 1:** When designing Dropbox's clients, which protocols or techniques should be used to efficiently monitor changes made by other clients? **Answer:** Listening to changes made by other clients can be achieved through push notifications using long polling or WebSockets, allowing real-time updates without the need for constant polling. When a client modifies a file, the synchronization server detects the change and pushes notifications to all relevant clients. This keeps clients updated instantly with minimal network overhead. In cases of unreliable networks or idle periods, the system may employ periodic polling to ensure eventual consistency. This combination ensures efficient synchronization while balancing network resource usage.

Data storage and replication

In Dropbox, data storage involves dividing files into smaller segments, or chunks, which are distributed across multiple storage servers for efficiency and scalability.
Metadata is maintained separately to record the location and identifiers of these chunks, allowing for fast lookup and retrieval. Each chunk is encrypted before storage to ensure data confidentiality. To ensure durability and high availability, Dropbox replicates data chunks across multiple data centers.
This replication provides redundancy and supports disaster recovery.
If one data center becomes unavailable due to hardware failure or network issues, data can still be retrieved from another without interrupting service. While replication strengthens fault tolerance, it also introduces challenges. The system must provide low-latency access for users worldwide, manage eventual consistency across replicas, and address the operational complexity and costs associated with managing data across geographically distributed servers.

Version control and conflict resolution

Dropbox allows multiple users to edit a document simultaneously, making version control and conflict resolution essential. The metadata server tracks file versions, storing previous copies for rollback if needed.
A simple strategy is last-write-wins, where the most recent upload overwrites earlier changes. To prevent data loss, Dropbox can create conflicted copies for simultaneous edits, appending timestamps or usernames to distinguish versions. For text-based files, merge strategies can automatically combine non-conflicting changes, though complex conflicts may still require manual resolution.
From a user perspective, the system notifies users of conflicts via the pub/sub system, allowing them to view and compare versions. The interface supports intuitive resolution, balancing automated handling with user control to manage conflicts efficiently without confusion.

Security mechanisms

Dropbox implements robust security measures to protect user data both at rest and in transit.
File chunks are encrypted using AES-256, while SSL/TLS ensures secure transmission, preventing interception or eavesdropping. The system also adheres to regulatory standards such as GDPR and HIPAA. GDPR compliance entails providing users with the ability to access, correct, and delete their data, as well as facilitating secure cross-border data transfers.
HIPAA compliance requires encrypting protected health information (PHI) and supporting Business Associate Agreements (BAAs) with healthcare providers.

4.5 Putting everything together

The Dropbox system integrates multiple components and services to support efficient file storage, synchronization, and sharing.

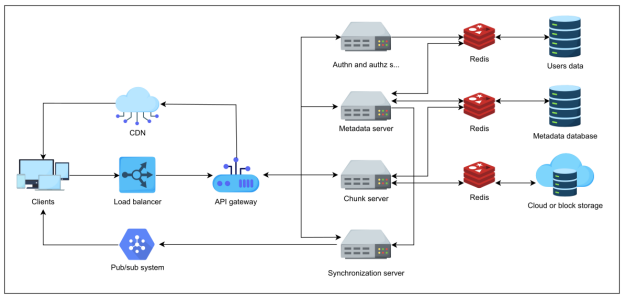


Figure 1.12: Detailed design of Dropbox system

Components overview:

- **Clients:** User devices with installed applications that interact with backend services to upload, download, and sync files.
- **Load balancer:** Distributes incoming client traffic evenly across servers for efficient resource use.
- **API gateway:** Entry point for client requests, routing them to the appropriate backend services.
- **CDN:** Delivers cached static content close to users to reduce latency.
- **Authentication and authorization service (Authn/Authz):** Verifies user identity and enforces access control.
- **File metadata server:** Maintains metadata (file names, paths, versions) and coordinates access to metadata databases and caches.
- **Metadata database:** Stores file metadata, including locations and access permissions, typically in a NoSQL database.
- **Chunk server:** Handles storage, retrieval, and management of file chunks efficiently.
- **Cloud or block storage:** Provides scalable storage for file chunks across distributed servers.
- **Synchronization server:** Tracks changes and ensures all clients have the latest file versions.
- **Pub/sub system:** Publishes updates to subscribed clients, enabling real-time synchronization and notifications.

  Note: User data and structured information, such as accounts or permissions, are stored in relational databases like MySQL.

4.6 Performance optimization

Dropbox employs several techniques to improve system performance and ensure efficient file transfers.
These methods include using redundant database servers, caching frequently accessed files, compressing file chunks during transfer, and applying data deduplication to avoid storing duplicate content. Network conditions—such as fluctuating bandwidth and latency—can significantly affect upload and download speeds. To mitigate these issues, Dropbox uses adaptive bandwidth control, multi-threaded transfers, and TCP optimizations that dynamically adjust transfer rates based on network quality.
Storing local copies of frequently accessed files also helps reduce retrieval time and latency. Compression reduces the amount of data transmitted, while deduplication ensures that only unique file chunks are uploaded, resulting in faster and more efficient synchronization.
Common compression algorithms include:

- Gzip (DEFLATE) for documents, JPEG for images, MP3 for audio
- LZ77 lossless compression
- On-the-fly algorithms such as LZMA, Snappy, and Brotli

For deduplication, Dropbox can use:

- SHA-256 hashing
- MD5
- Rabin Fingerprinting
- Content-defined chunking (CDC)

These strategies collectively enhance throughput, reduce storage costs, and improve user experience across devices.

4.7 Conclusion

The Dropbox design strikes a balance between functional efficiency and scalable architecture.
File synchronization, version control, and access management are supported through chunking, hashing, and metadata management, ensuring reliability at scale. High availability, fault tolerance, and performance optimizations, including real-time updates and deduplication, provide consistent and efficient user experiences across devices.

5 Ticketmaster System Design

High-demand ticketing platforms, such as those used for major concerts or events, must handle millions of users attempting to access the same seats simultaneously.
These systems facilitate event searches, reservations, online payments, and digital ticket delivery, providing a streamlined user experience. Developing such platforms is a complex engineering challenge. Systems must manage extremely high traffic volumes while ensuring fairness, minimizing errors, and maintaining low latency.
Designing a scalable online ticketing system, like Ticketmaster, involves addressing multiple technical challenges. These systems must efficiently handle millions of concurrent requests, making the design both intricate and instructive.

This lesson examines the System Design of Ticketmaster, encompassing functional requirements, capacity estimations, API design, and the overall architecture of the complete system. We will begin by examining the core requirements for building a robust and scalable ticketing platform.

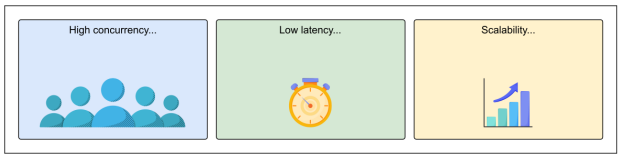


Figure 1.13: Challenges in designing online ticket booking systems

5.1 Requirements for Ticketmaster

Defining a system's requirements is the first step in structuring a well-defined architecture. Requirements establish the core functionalities and performance expectations that guide subsequent design decisions. These can be categorized into functional and non-functional requirements.
A clear understanding of both categories is essential for anticipating user needs and system resilience under high load, particularly in high-traffic scenarios like ticket sales

Functional requirements

Key functionalities for a robust online ticketing system include:

- **List cities and venues:** Provide users with up-to-date catalogs of cities and venues, ensuring reliable and accessible data.
- **Search for events:** Enable efficient event search using filters such as date, location, or genre.
- **Reserve and purchase tickets:** Allow users to select seats and complete transactions seamlessly.
- **Manage bookings:** Enable users to view, modify, or cancel bookings efficiently.
- **Generate and validate tickets:** Issue digital tickets upon purchase and validate them securely at entry to prevent fraud.

Nonfunctional requirements

Critical non-functional requirements for a scalable ticketing system include:

- **Availability:** Ensure the system is highly available to meet user demand.
- **Scalability:** Handle millions of concurrent users, particularly during high-demand events.
- **Performance:** Provide low-latency responses, especially during seat selection.
- **Reliability:** Prevent double-booking or seat overselling.
- **Fault tolerance:** Maintain operation with minimal impact in case of server failures.
- **Security:** Protect user data, prevent fraud, and secure payment processing.

Effective scalability is essential; without it, the system may fail under heavy load during popular events. Accurately estimating traffic and ensuring resources can dynamically adjust is critical for maintaining both performance and user experience.
Balancing functional and nonfunctional requirements often involves trade-offs. Using structured trade-off matrices can help determine where compromises are acceptable without impacting user satisfaction or system stability.
Next, we will examine how to approach scalability considerations to ensure robust system performance.

5.2 Estimating resources for Ticketmaster

Once system requirements are established, the next step in System Design is to estimate the resources required to handle peak user demand efficiently.
This assessment determines the infrastructure necessary to maintain performance and scalability. Specifically , it involves calculating the number of application servers and evaluating how the system can sustain high traffic without degrading the user experience.
A key aspect of this process is estimating the number of application servers needed to maintain performance under peak load. By analyzing potential concurrent users and average request rates, it is possible to determine the server capacity required to ensure the system scales effectively.

Servers estimation for Ticketmaster

Assumptions:

- The system has 100 million daily active users (DAUs).
- Each user generates 100 requests per day during peak periods.
- Each server can handle approximately 64,000 requests per second (RPS).

Using these assumptions:

$$\text{Reqs}_{/\text{day}} = \text{DAUs} \times \text{Req/user} = 100\text{ M} \times 100 = 10\text{ Billion}_{/\text{day}}$$

$$\text{Reqs}_{/\text{sec}} = \frac{10\text{ Billion}}{86,400} \approx 115\text{ K}_{/\text{sec}}$$

$$\text{Servers} = \frac{\text{Req}_{/\text{sec}}}{\text{RPS of a server}} = \frac{115\text{ K}}{64,000} \approx 2\text{ Servers}$$

These calculations indicate the minimum number of servers required to support typical daily traffic. However, during a traffic surge, when many users generate requests simultaneously, the peak request rate can approach the total number of DAUs:

$$\text{Servers}_{\text{peak}} = \frac{100\text{ M}}{64,000} \approx 1562\text{ Servers}$$

These estimates are suitable for an interview context but can be refined further for production planning. For a detailed approach, refer to the back-of-the-envelope calculations.

Storage estimation for Ticketmaster

In this section, we estimate the storage requirements for a ticketing system.
Assumptions:

- The system serves 1,000 cities.
- Each city has 5 cinemas.
- Each cinema has approximately 1,000 seats.
- Each cinema hosts 2 shows per day on average.

A possible schema to store a booking is as follows:

```table_schema
TABLE Booking(
  ID int, # 4 bytes
  Num_Seats int, # 4 bytes
  Show_ID int, # 4 bytes
  Movie_ID int, # 4 bytes
  Seat_Numbers varchar(50), # 50 bytes
  Booking_Time timestamp, # 4 bytes
)
```

This results in approximately 70 bytes per booking.

$$\text{Storage}_{/\text{day}} = 1,000 \text{ cities} \times 5 \text{ cinemas} \times 1,000 \text{ seats} \times 2 \text{ shows} \times 70 \text{ bytes} = 700 \text{ MB}/\text{day}$$

$$\text{Storage}_{/\text{month}} = 700 \text{ MB} \times 30 \approx 21 \text{ GB}$$

Under this configuration, roughly 256 GB of storage would suffice for a year's data, which is manageable for a large-scale service.

> **Note:** In a production system, multiple tables would exist for user accounts, transaction history, and inventory management. This calculation focuses on core booking storage requirements.

Practical considerations:

1. **Over-preparation:** Underestimating peak loads can create scalability issues. It is essential to consider unexpected surges, such as high-demand events, when planning infrastructure.
2. **Efficient data management:** Implement archiving strategies for outdated data and leverage cost-effective long-term storage to optimize resources.
3. **Real-world testing:** Conduct stress tests that simulate peak traffic to validate estimates and identify bottlenecks, enabling proactive tuning of the system.

Designing key API endpoints is the next step in ensuring the system operates efficiently and communicates effectively across components.

5.3 API design for Ticketmaster

A well-defined API enables efficient communication between clients and the Ticketmaster system. Key API functionalities include:

- **Search:** Returns available movies and showtimes for a specified location and time. Example request:

```
-- Search GET /search?city=Seattle&cinema=Regal&time=7pm
```

- **Bookings and payments:** Temporarily reserves seats while the user completes payment. Example request:

```
-- Reserve Seats POST /create
Body: { "showId": "s2411",
"userId": "u1412", "seats": ["A1", "A2"]
}
```

- **Booking confirmation:** Confirms the reservation after payment has been processed. Example request:

```
-- Confirm Booking
POST /confirm
Body: {
    "reservationId": "r2803",
    "paymentInfo": {...}
}
```

- **Retrieve booking:** Retrieves booking details for user reference or customer service. Example request:

```
-- Get Booking GET /booking?bookingId=b0102
```

- **Booking cancellations:** Cancels an existing booking and releases the reserved seats. Example request:

```
-- Cancel Booking DELETE /booking?bookingId=b0102
```

> **Quiz: Question 1:** What transport and communication protocols are suitable for
>
> Ticketmaster's APIs, and why? **Answer:** Ticketmaster's APIs can use REST for stateless, standard client-server interactions, providing simplicity and compatibility. For real-time updates, WebSockets are appropriate because they allow low-latency,
>
> bidirectional communication, which is crucial for live ticket availability and minimizing double-booking risks. **Question 2:** Should Ticketmaster use an SQL or NoSQL database, and why? **Answer:** SQL databases (e.g., PostgreSQL, MySQL) are suitable for transactional operations like bookings because they provide strong consistency guarantees. NoSQL databases (e.g., MongoDB, Cassandra) are useful for caching real-time availability and handling high read loads. A hybrid approach, using SQL for transactional data and NoSQL for fast lookups or caching, is often the most effective solution.

Defining these endpoints provides a clear foundation for System Design. The architecture will also require monitoring, load balancing, and other core components to ensure reliability and scalability.

5.4 Building blocks

Designing a scalable Ticketmaster service requires several core components:

- **Load balancers:** Distribute incoming traffic across multiple servers to ensure system stability and prevent overload.
- **Databases:** Store information about cinemas, movies, cities, and users. SQL databases (e.g., MySQL, PostgreSQL) are preferred for their ACID compliance. Certain tasks may benefit from NoSQL databases to improve performance and scalability.
- **Web servers:** Serve static content, handle client requests, and forward dynamic requests to application servers.
- **Application servers:** Execute business logic, process API requests, and interact with databases.
- **Cache:** Improve response times by storing frequently accessed data.
- **Monitoring:** Track system health, detect anomalies, and log events for debugging and analysis.
- **Blob stores:** Store multimedia content such as movie posters or trailers.

Practical insight: Selecting the appropriate database technology is critical. SQL databases provide strong transactional support, while adding NoSQL databases for specific tasks can enhance scalability and performance. Choosing the right technology for each use case ensures the system remains efficient under high demand.

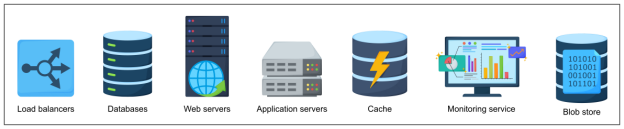


Figure 1.14: The building blocks used in the System Design of Ticketmaster

5.5 Design of Ticketmaster

With the core building blocks identified, we can proceed to the System Design of Ticketmaster. This section provides a high-level overview followed by considerations for detailed design.

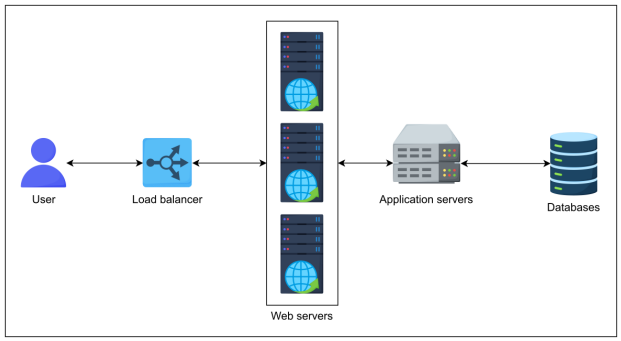


Figure 1.15: High-level design of Ticketmaster

In this design, the load balancer distributes incoming requests to web servers, which serve the client-facing application. Requests requiring business logic are forwarded to application servers, which process the operations and interact with the databases to store or retrieve relevant information.
This architecture addresses the system's functional requirements. Achieving non-functional goals such as scalability, performance, and security requires careful attention to redundancy, fault tolerance, and efficient data management.
Integrating these considerations ensures a resilient ticketing platform that can support high volumes of concurrent users.

Detailed System Design of Ticketmaster

Expanding on the high-level design, we can identify services that fulfill both functional and nonfunctional requirements.

Users begin by requesting show searches, ticket bookings, and payments.
Load balancers efficiently manage these requests by distributing traffic across multiple web servers, preventing bottlenecks and enabling horizontal scaling by adding additional servers as required. All requests are routed through an API gateway, which provides a centralized entry point for client interactions.
Web servers serve static content securely, manage user sessions, and forward requests to application servers, which implement the business logic across several microservices:

- **Ticket service:** Handles reservations and ticket issuance.
- **Event service:** Manages large-scale event-related functionalities, including high-demand ticket releases, real-time seat availability updates, and event-based notifications.
- **User service:** Manages user authentication, profiles, and account settings.
- **Location service:** Stores and retrieves cinema-related data, including locations and available screens.
- **Search service:** Processes queries for movies, cinemas, and showtimes.

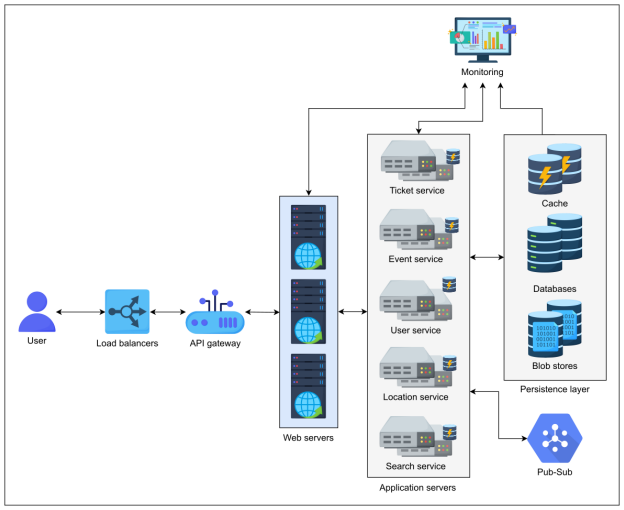


Figure 1.16: Detailed design of Ticketmaster

The application servers interact with the persistence layer to store, retrieve, and cache data efficiently.
A publish-subscribe system facilitates real-time event handling, such as notifications and updates for high-demand ticket releases. To ensure reliability and performance, a monitoring system continuously tracks system health, providing real-time analytics and alerts for proactive maintenance.
Ticketmaster is distinguished by its ability to handle high concurrency and real-time demands during major ticket releases.
Unlike typical System Designs, its architecture must manage extreme traffic spikes while maintaining low latency and high availability, ensuring a seamless user experience. Let 's now examine what separates Ticketmaster from typical System Design problems.

5.6 Unique aspects of Ticketmaster's design

Designing Ticketmaster introduces challenges that distinguish it from typical System Design problems, primarily due to requirements for extreme scalability and high concurrency. Key aspects of its architecture include:

- **Concurrency handling:** Efficiently managing simultaneous seat bookings to prevent overselling and conflicts.
- **High availability:** Maintaining system reliability and uptime, particularly during high-traffic events.
- **Security:** Protecting against unauthorized access, fraud, and cyber threats.

Each of these aspects is essential to providing a seamless and secure ticketing experience. The following sections will examine these considerations in detail.

Concurrency handling

A significant challenge in Ticketmaster's system is managing concurrent seat booking requests to avoid conflicts and prevent overselling. Common approaches to address these challenges include:

1. **Optimistic locking:** Suitable when conflicts are infrequent. The system monitors seat records using version numbers. If the version has changed during a booking attempt, the transaction is rolled back, indicating that another user has successfully booked the seat. This approach is less restrictive but may result in more transaction failures under high competition.

2. **Pessimistic locking:** The system locks the seat record when a booking begins, preventing other transactions from accessing it until the lock is released. This method ensures that no two users can book the same seat simultaneously, although it may reduce throughput under heavy load due to queuing.

3. **Distributed locking:** In a distributed environment, a centralized lock manager (e.g., Redis or Zookeeper) ensures consistent access to seat records across multiple servers. This prevents conflicts regardless of where requests originate. While essential for scalability, it introduces additional system complexity.

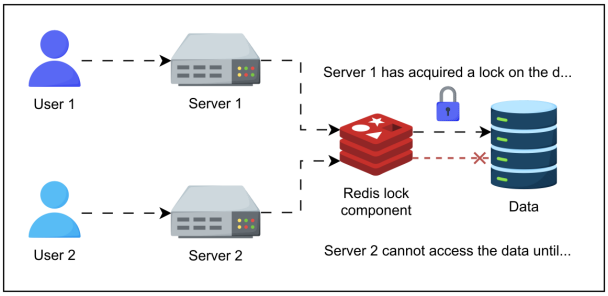


Figure 1.17: Distributed locking in action

Practical insight: Managing concurrency requires careful balance.
Optimistic locking can be efficient, but it is important to assess the likelihood of conflicts in the specific system context. Designing for worst-case scenarios and conducting pilot tests during live events can prevent unexpected failures and ensure reliable operation.

High availability

The Ticketmaster system must maintain fault tolerance and high availability to ensure a seamless user experience, even in the event of server or network failures. Key strategies to achieve these goals include:

- **Redundancy:** Deploying multiple instances of each system component, such as web servers, application servers, and databases, ensuring traffic is distributed and that failures of individual servers do not disrupt service.

- **Replication:** Creating multiple copies of databases across different servers or data centers, allowing the system to continue operations seamlessly if a database fails.

- **Failover:** Implementing automatic failover mechanisms, redirecting traffic to standby servers in real time to minimize downtime and maintain user experience.

Integrating these strategies allows the system to handle unexpected failures robustly, maintaining high availability and reliability. These practices have proven effective in real-world applications, addressing availability challenges.

> **Note:** Numerous additional strategies exist for ensuring high availability, including autoscaling, graceful degradation, chaos engineering, and backup mechanisms. This section introduces the fundamental approaches.

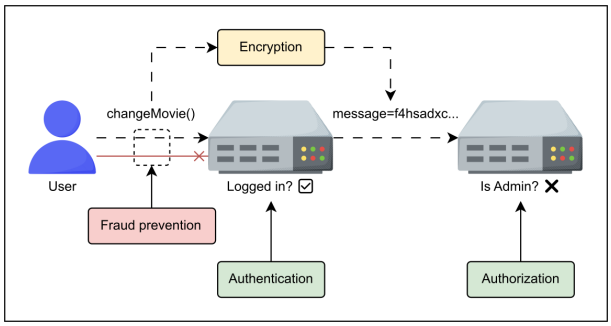
Security

Unauthorized access and data breaches pose significant risks to Ticketmaster, potentially compromising sensitive user data, including personal information, payment details, and booking history.
High-profile ticketing platforms are vulnerable to various threats, including credential stuffing, phishing attacks, payment fraud, automated bot-driven scalping, and denial-of-service (DoS) attacks. Implementing

robust security measures is crucial for maintaining user trust, ensuring regulatory compliance, and preserving system integrity.

Solutions

- **Authentication and authorization:** Implementing role-based access control (RBAC) for users and administrators, ensuring that only authorized personnel can access specific functionalities or data and limiting exposure to sensitive information.
- **Data encryption:** Encrypting sensitive data, particularly payment-related information, protects it from unauthorized access both in transit and at rest.
- **Fraud prevention:** Deploying systems to detect and prevent suspicious activities and duplicate bookings, identifying patterns indicative of fraud using machine learning algorithms to maintain transaction integrity.



The illustration highlights key security techniques implemented in Ticketmaster.
It authenticates users to verify identities, authorizes actions to restrict administrative privileges, and prevents fraud to block unauthorized activities, such as ticket scalping or API misuse. Encryption ensures secure communication by encoding sensitive messages, collectively safeguarding the system from unauthorized access, fraudulent activities, and data breaches.
Multiple strategies exist for securing a system like Ticketmaster, including adherence to compliance standards, robust network security, proactive monitoring, and effective threat mitigation.
This section highlights key measures, including PCI-DSS compliance for secure payments, DDoS protection through firewalls and API security, and regular security audits to identify and prevent vulnerabilities. These measures collectively ensure data protection, system integrity, and resilience against evolving cyber threats.
Practical insight: Prioritizing security early in System Design is critical. Implementing thorough authentication protocols, robust encryption, access controls, and regular security audits has proven essential for preventing data breaches and maintaining user trust.

5.7 Conclusion

Designing a distributed system such as Ticketmaster requires balancing scalability, reliability, security, and high performance.
This discussion examined essential design considerations, including API architecture, database management, security enforcement, and handling high concurrency. Together , these components determine the system's ability to manage large-scale ticketing events efficiently.
Future developments in AI-driven dynamic pricing, blockchain-based fraud prevention, and cloud-native architectures can further enhance system resilience and elasticity. Advancements in predictive analytics and real-time event monitoring may improve operational efficiency while ensuring fair access to high-demand events.